

Scaling Up Knowledge Graph Creation to Large and Heterogeneous Data Sources

Paper Summary, Implementation Steps, Downstream tasks

1 Introduction

KGs have become fundamental structures for organizing and querying complex, interconnected data across domains ranging from biomedicine to enterprise knowledge management. However, their construction from heterogeneous sources remains computationally prohibitive at scale. Traditional RML engines suffer from inefficient execution strategies, often leading to memory exhaustion when processing large datasets with complex join operations.

We address this challenge by implementing an optimized KG creation pipeline based on the methodology presented by Iglesias et al. [1]. We build on the RML-Planner framework [2] to process genomic datasets sourced from the SDM-Genomic repository [3], demonstrating significant improvements in execution efficiency through intelligent mapping partitioning and bushy tree execution plans.

1.1 Problem Statement

Standard RML materialization engines (SDM-RDFizer [6], Morph-KGC [5], RMLMapper) typically execute mapping assertions in isolation or with naive parallelization. When confronted with datasets containing millions of records and mappings requiring multi-way joins, these approaches exhibit three critical limitations:

1. **Redundant I/O:** Source files are repeatedly loaded for each mapping assertion, even when multiple assertions reference the same source.
2. **Intermediate Result Explosion:** Join operations produce large intermediate datasets, with duplicates only removed at the final materialization stage.
3. **Sequential Bottlenecks:** Dependent mappings cannot be parallelized effectively, creating execution bottlenecks.

The RML-Planner addresses these issues through a cost-based optimization framework that treats KG creation as a query optimization problem, analogous to relational database query planning.

2 Theoretical Foundation

2.1 The Planning Algorithm

The core contribution of Iglesias et al. [1] lies in transforming the KG creation problem into an optimization task over a hypergraph representation of mapping dependencies.

2.1.1 Mapping Hypergraph Construction

Given a set of RML mapping assertions $M = \{m_1, m_2, \dots, m_n\}$ and data sources $S = \{s_1, s_2, \dots, s_k\}$, the planner constructs a hypergraph $H = (V, E)$ where:

- Each vertex $v_i \in V$ represents a mapping assertion m_i
- A hyperedge $e \in E$ connects vertices sharing either:
 1. The same logical source (intra-source dependency)
 2. A parent-child relationship via `rr:parentTriplesMap` (inter-source join)

2.1.2 Partition Identification

The planner performs hypergraph partitioning to group related mappings into execution units. The partitioning strategy distinguishes:

Intra-Source Partitions (P_{intra}): Mappings accessing a single source s_i . These execute as efficient *identifying projections*—a single scan of the source produces all required triples for the group. For example, if three mapping rules all reference `patients.csv`, they execute together in one pass.

Inter-Source Partitions (P_{inter}): Mappings requiring joins between sources. The planner identifies common join attributes and groups mappings to minimize the number of join operations. Rather than executing n independent joins, mappings with shared join predicates are merged into a single multi-way join.

2.1.3 Bushy Tree Construction

Unlike traditional left-deep or right-deep join trees, the planner constructs bushy trees where independent partitions execute in parallel. The algorithm proceeds as follows:

1. Identify independent partitions (no shared sources or join dependencies)
2. Assign each partition to a parallel execution thread
3. For dependent partitions, construct a partial order based on data flow
4. Insert **eager duplicate removal** (DR) operators at merge points

The key innovation is the placement of DR operators. Traditional engines apply deduplication only at the end, after all triples are materialized. The planner inserts DR immediately after each partition’s execution, before merging results. This drastically reduces intermediate dataset cardinality.

2.1.4 Quantified Benefits

The paper demonstrated the following improvements on benchmark datasets:

- **Execution Time:** Reduction of 40-70% compared to sequential execution on datasets exceeding 10M triples
- **Memory Footprint:** Up to $3\times$ reduction through eager duplicate removal
- **Scalability:** Successful materialization of datasets that previously caused memory overflow (OOM) failures in baseline engines

For instance, on the BSBM benchmark with 100M triples, the sequential RMLMapper approach required 18GB RAM and 47 minutes, while the planner-optimized execution completed in 19 minutes with only 6GB RAM.

3 Implementation

We now describe our end-to-end implementation workflow for KG materialization. Our goal is to process genomic mutation data from heterogeneous sources (CSV and JSON) into a unified, queryable graph structure.

3.1 Execution Pipeline Overview

1. **Loading dependencies:** rdflib, SDM-RDFizer, cerebras-cloud-sdk
2. **Mapping Design:** Crafting RML rules that normalize heterogeneous schemas into a unified ontology
3. **Planner Execution:** Invoking the planning algorithm to generate an optimized execution plan, then materializing the KG via SDM-RDFizer
4. **Validation:** Executing SPARQL queries and RAG-based QA to verify correctness and utility

3.2 Dataset Acquisition

We utilized the SDM-Genomic dataset [3], which provides cancer genomics data with the following format:

A	B	C	D	E	F	G	H	I	J	K	L	M	N
Gene name	Accession Number	Gene CDS length	HGNC ID	ID_Sample	ID_Synced	Primary site	Primary histology	Mutation ID	Mutation Description	Mutation genome position	Mutation somatic status	SubstEMD	cFormat
KIT	ET0000008125.5	2931	6342	1072079	907353	Soft_tissue	Gastrointestinal_stomach	24165453	SubstEMD-missense	4.55553410-55553410	Confirmed somatic variant	18297704	KIT_c.1678T>G
TRK3	ET00000049105.2	2172	11602	1763115	1687414	Endometrium	Carcinoma	31144031	SubstEMD-missense	12115106652-151106652	Reported in another cancer sample as somatic	TRK3_c.1986G>A	
PHLID3	ET00000036387	845	28570	2746125	2604820	Large_intestine	Carcinoma	26597877	Unknown	23105461973-105461973	Confirmed somatic variant	PHLID3_c.227-121del	
SLC3A1	ET00000043469	2079	10952	1760179	1572453	Breast	Carcinoma	45247080	Unknown	15492570445-42570445	Confirmed somatic variant	SLC3A1_c.647-400T>C	
BAP1	ET00000026388	2136	950	1780577	1684576	Lung	Carcinoma	21956184	Unknown	352449372-52449372	Confirmed somatic variant	BAP1_c.660-345-A	
ESR1	ET00000043389.2	2046	25966	2680291	2547253	Prostate	Carcinoma	49898980	Unknown	879589997-9589997	Confirmed somatic variant	ESR1_c.1852-244G>A	
ERG	ET00000038910.1	1392	3446	2192372	2064500	Ovary	Carcinoma	42726456	Unknown	3139993995-39993995	Confirmed somatic variant	ERG_c.149-371del-T	
CNTBL	ET00000038341.5	6978	1859	2662366	2522490	Lung	Carcinoma	23436076	SubstEMD-missense	9123912552-123912552	Confirmed somatic variant	27623066	CNTBL_c.3754G>C
TMF8	ET00000038312.2	1788	11023	2296303	2161890	Upper_aerodigestive	Carcinoma	26028465	SubstEMD-missense	112171729-12171729	Reported in another cancer sample	25275208	TMF8_c.880C>T
FGFR1	ET00000035922	2439	3688	1723061	1638883	Central_nervous_system	Primitive_neuroectodermal	26158207	Unknown	838278622-38278622	Confirmed somatic variant	22722829	FGFR1_c.754T>C
TMF2	ET00000056864.3	763	11096	1651144	1565931	Large_intestine	Carcinoma	31460272	Unknown	922968618-12968618	Confirmed somatic variant	22833696	TMF2_c.61-615G>T
DHR9	ET000000412271	960	18888	2635121	2485517	Endometrium	Carcinoma	42872339	Unknown	2169938045-169938045	Confirmed somatic variant	DHR9_c.47T>C	
PAS3	ET00000028080.3	1785	9081	2395255	2246897	Neuroendocrine_and_lymphoid_neoplasm		20932727	Unknown	28114881057-114881057	Confirmed somatic variant	21842062	PAS3_c.309-1252G>A
FGFR1	ET000000397113.2	2463	3688	2658286	2538445	Large_intestine	Carcinoma	38955149	SubstEMD-missense	83828661-3828661	Confirmed somatic variant	27148462	FGFR1_c.718A>G
PARD8	ET000000406010	3638	14446	2197184	2095422	Pancreas	Carcinoma	41506207	Unknown	2205474705-205474705	Confirmed somatic variant	PAR88_c.120-6083C>T	
MYC	ET00000077879.2	1395	7553	1818103	1718247	Hepatoblastic_and_lymphoid_neoplasm		36939606	SubstEMD-missense	812870661-18770663	Confirmed somatic variant	MYC_c.220C>T	
CDKN2A	ET000000304884	471	1767	2726010	2544831	Oropharynx	Carcinoma	27316379	SubstEMD-missense	9121971111-121971111	Confirmed somatic variant	28481389	CDKN2A_c.247C>T
STGBR1	ET000000549511.1	633	16666	1892119	1751480	Breast	Carcinoma	75247080	Unknown	142032256-2032256	Confirmed somatic variant	STGBR1_c.39-1000A>C	
FM05	ET000000369272	858	3773	2607098	2389434	Pancreas	Carcinoma	35075318	Unknown	1146671913-146671913	Confirmed somatic variant	FM05_c.830-850G>A	
IGHMBP1	ET000000592824	578	5642	2658327	2534948	Large_intestine	Carcinoma	73903087	Unknown	115857658-6857658	Confirmed somatic variant	27148462	IGHMBP1_c.257Tdel
ZNF93	ET00000043789.5	1863	13169	2197667	2065945	Kidney	Carcinoma	28454917	Unknown	1920043000-20043000	Confirmed somatic variant	ZNF93_c.227-931A>T	
KMT5C	ET000000362189.6	14736	13776	2296294	2181897	Upper_aerodigestive	Carcinoma	21549699	SubstEMD-missense	715192705-15192705	Reported in another cancer sample	25275208	KMT5C_c.296T>C
PRF20	ET00000072247.2	1701	6552	2186510	2654807	Ovary	Carcinoma	34864405	SubstEMD-missense	146877623-46877623	Confirmed somatic variant	PRF20_c.54C>A	
ACSRG2	ET00000056989.1	2001	24174	2633830	2484205	Breast	Carcinoma	82167297	SubstEMD-missense	191837880-8187880	Confirmed somatic variant	ACSRG2_c.1751G>A	
EPAPH2	ET00000055827	3251	9877	2134688	2063260	Lung	Carcinoma	32266987	Unknown	239618405-9618405	Confirmed somatic variant	EPAPH2_c.1355-140A>A	
LY75	ET000000534824	5454	6729	1766805	1671125	Large_intestine	Carcinoma	73205881	SubstEMD-missense	2160732115-10732115	Confirmed somatic variant	22895193	LY75_c.1814C>A
ILP2	ET00000042884.2	2271	30248	2124527	1948473	Liver	Carcinoma	47099129	SubstEMD-missense	1833736236-33736236	Confirmed somatic variant	ILP2_c.1051A>G	
CD22	ET00000070311	1899	1643	1842442	1726707	Pancreas	Carcinoma	22850935	Unknown	193583526-3583526	Confirmed somatic variant	CD22_c.1676-446G>A	
SDC6	ET00000049491	423	11138	2744887	2603930	Lung	Carcinoma	38903037	Unknown	49080516-9080516	Confirmed somatic variant	SDC6_c.307-148G>T	

- **75percent_of_records_with_duplicate_and_each_duplicate_being_repeated_10times.csv:**
75% of unique records, with intentional duplications (10× replication per record) to simulate real-world data redundancy
- **25percent_of_records_with_duplicate_and_each_duplicate_being_repeated_10times.json:**
The remaining 25%, in JSON format

Each record describes a genomic variant with attributes: Gene name, HGNC ID, Mutation Description, cFormat, Sample ID, Primary Site, and Primary Histology. The heterogeneity (CSV vs JSON) and high duplication factor (10×) stress-test the planner's deduplication and source abstraction capabilities.

3.3 Core Component Adaptation

The RML-Planner codebase [2] required several modifications for compatibility with modern Python environments and our specific use case.

3.3.1 Planning Logic (planner/planning.py)

This module orchestrates the entire pipeline. Its workflow is:

1. **Parse RML Mappings:** Load the `.ttl` file via `rdflib.Graph.parse()`.
2. **Extract Triples Maps:** Query the parsed graph for `rml:logicalSource`, `rr:subjectMap`, and `rr:predicateObjectMap` nodes, constructing internal `TriplesMap` objects.
3. **Classify Mappings:** Iterate through `TriplesMap` instances to identify dependencies. If a mapping references `rr:parentTriplesMap`, it requires a join; otherwise, it's an independent projection.
4. **Invoke Partitioner:** Call `partitions_classification()` from `functions.py` to group mappings.
5. **Generate Execution Plan:** Construct the bushy tree and schedule partition execution order.

3.3.2 Execution Management (planner/functions.py)

This utility module performs physical execution. Key functions:

`config_writer(...)`: Dynamically generates `.ini` configuration files for each partition. For example, if Partition 1 contains only `Gene_CSV` mappings, it generates a minimal config pointing to just the CSV file and the subset of relevant mapping rules.

`execute_partitions(...)`: Spawns subprocesses to run `SDM-RDFizer` on each partition.

3.3.3 Mapping Abstraction (planner/triples_map/TriplesMap.py)

The `TriplesMap` class encapsulates an RML mapping rule. It exposes attributes like:

- `data_source`: Path to the logical source file
- `reference_formulation`: Format specifier (`ql:CSV` or `ql:JSONPath`)
- `subject_map`: Template for generating subject URIs
- `predicate_object_maps_list`: Array of predicate-object pairs

The planner inspects these attributes to determine mapping complexity and cost.

3.4 Unified Mapping Strategy

We designed a single `.ttl` file to handle both data sources, ensuring schema normalization. This is heavily inspired from the RML mappings for gene entities defined in [4, 7]

```
1 # Define Gene entities from CSV source
2 <#Gene_CSV>
3   # Logical source definition
4   rml:logicalSource [
5     rml:source "75
percent_of_records_with_duplicate_and_each_duplicate_being_repeated_10times.
csv";
6     rml:referenceFormulation ql:CSV
7   ];
8
9   # Subject definition: Generate URIs like http://project-iasis.eu/Gene/TP53
10  rr:subjectMap [
11    rr:template "http://project-iasis.eu/Gene/{Gene name}";
```

```

12     rr:class iasis:Gene # Assign rdf:type
13 ];
14
15 # Predicate-Object: Gene label
16 rr:predicateObjectMap [
17     rr:predicate iasis:geneLabel;
18     rr:objectMap [ rml:reference "Gene name" ] # Extract from CSV column
19 ];
20
21 # Predicate-Object: HGNC identifier
22 rr:predicateObjectMap [
23     rr:predicate iasis:hgncID;
24     rr:objectMap [ rml:reference "HGNC ID" ]
25 ].

```

Listing 1: Unified RML Mapping for Gene Entities

Explanation: This mapping instructs the engine to:

1. Read the CSV file row by row
2. For each row, generate a Gene URI using the "Gene name" column value (e.g., `Gene/TP53`)
3. Assert two triples: (`Gene/TP53`, `rdf:type`, `iasis:Gene`) and (`Gene/TP53`, `iasis:geneLabel`, `"TP53"`)

We replicate this pattern for JSON, using `ql:JSONPath` with an iterator `$[*]` to traverse array elements. Critically, both CSV and JSON genes map to the same URI template (`http://project-iasis.eu/Gene/{Gene name}`), ensuring automatic deduplication—if "TP53" appears in both files, only one `Gene/TP53` node is created.

Inter-source joins are defined via `rr:parentTriplesMap`:

```

1 <#Variation_CSV>
2 ...
3 rr:predicateObjectMap [
4     rr:predicate iasis:variation_isLocatedIn_gene;
5     rr:objectMap [
6         rr:parentTriplesMap <#Gene_CSV>;
7         rr:joinCondition [
8             rr:child "Gene name"; # Column in Variation table
9             rr:parent "Gene name"; # Column in Gene table
10        ];
11    ];
12 ].

```

Listing 2: Join Condition for Variation-to-Gene Link

This instructs a join: match Variation records with Gene records where `Variation[Gene name] == Gene[Gene name]`, then create an edge from the Variation's URI to the Gene's URI.

3.5 Execution

Running `python3 planning.py config.planning_qa.ini` triggers the pipeline. The planner:

1. Parses `mapping_planning_qa.ttl`, identifying 6 triples maps (3 CSV, 3 JSON)
2. Groups them into partitions (e.g., all CSV Gene/Variation/Sample maps form one partition)
3. Generates partition-specific configs
4. Invokes SDM-RDFizer on each partition in parallel

5. Merges outputs, applying eager duplicate removal

Execution Results:

- **Input Scale:** CSV file with $\sim 100,000$ records (75% partition, $10\times$ duplication), JSON with $\sim 33,000$ records
- **Output KG: 34,492 unique triples**
- **Deduplication Effect:** Despite 133,000 input records with $10\times$ replication, only 34K triples were materialized, demonstrating effective duplicate removal
- **Execution Time:** 5.2 seconds (measured on Intel i5, 16GB RAM)

Representative output triples:

```
1 <http://project-iasis.eu/Gene/A1CF> <http://project-iasis.eu/vocab/geneLabel> "
  A1CF" .
2 <http://project-iasis.eu/Gene/A1CF> <http://project-iasis.eu/vocab/hgncID>
  "24086.0" .
3 <http://project-iasis.eu/Variation/40238221> <http://project-iasis.eu/vocab/
  cFormat> "A1CF_c.?" .
4 <http://project-iasis.eu/Variation/40238221> <http://project-iasis.eu/vocab/
  mutationDescription> "Deletion-frameshift" .
5 <http://project-iasis.eu/Variation/40238221> <http://project-iasis.eu/vocab/
  variation_isLocatedIn_gene> <http://project-iasis.eu/Gene/A1CF> .
6 <http://project-iasis.eu/Sample/2506626> <http://project-iasis.eu/vocab/
  sampleID> "2506626" .
7 <http://project-iasis.eu/Sample/2506626> <http://project-iasis.eu/vocab/
  primarySite> "haematopoietic_and_lymphoid_tissue" .
```

The successful generation of only 34K triples from 133K records validates the planner's deduplication efficacy.

4 Downstream Task 1: Structured QA Evaluation

4.1 Objective and Methodology

To assess the KG's semantic correctness, we designed a systematic evaluation using SPARQL, the standard query language for RDF graphs. Our goal was to verify:

1. **Structural Integrity:** Are entities correctly linked (e.g., Mutations connected to Genes)?
2. **Query Expressiveness:** Can the graph answer complex multi-hop queries?
3. **Data Quality:** Are there orphan nodes or missing attributes?

We implemented `qa_evaluation.py`, which loads the KG using `rdflib` and executes 14 predefined SPARQL queries spanning:

- **Entity Retrieval:** Direct attribute lookup (e.g., "What is the HGNC ID of KIT?")
- **Path Traversal:** Following relationships (e.g., "Find samples containing EGFR mutations")
- **Aggregation:** Counting related entities (e.g., "How many mutations does KRAS have?")
- **Filtering:** Pattern matching on properties (e.g., "Find deletions using regex")

4.2 Implementation Details

Each query follows this template:

```
1 PREFIXES = ""
2 PREFIX iasis: <http://project-iasis.eu/vocab/>
3 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
4 ""
5
6 query = PREFIXES + ""
7 SELECT ?label ?hgncID
8 WHERE {
9     ?gene a iasis:Gene ;
10         iasis:geneLabel "KIT" ;
11         iasis:hgncID ?hgncID .
12 }
13 ""
14
15 results = graph.query(query)
16 for row in results:
17     print(row)
```

Listing 3: SPARQL Query Execution Pattern

The script systematically tests all 14 queries and outputs results for manual inspection.

4.3 Results and Analysis

Query 1: Gene Attribute Retrieval

```
1 Q: What is the label and HGNC ID of the gene 'KIT'?
2 A:
3 (rdflib.term.Literal('KIT'), rdflib.term.Literal('6342.0'))
```

Interpretation: The query successfully retrieved the HGNC identifier for KIT (gene ID 6342), confirming that Gene entities are correctly materialized with both label and identifier properties.

Query 2: Relationship Traversal

```
1 Q: List 5 mutations found in the gene 'TP53'.
2 A:
3 (rdflib.term.Literal('TP53_c.524G~A'), rdflib.term.Literal('Substitution-
4 missense'))
5 (rdflib.term.Literal('TP53_c.856G~A'), rdflib.term.Literal('Substitution-
6 missense'))
7 (rdflib.term.Literal('TP53_c.406C~G'), rdflib.term.Literal('Substitution-
8 missense'))
9 (rdflib.term.Literal('TP53_c.865C~T'), rdflib.term.Literal('Substitution-
10 missense'))
11 (rdflib.term.Literal('TP53_c.655C~T'), rdflib.term.Literal('Substitution-
12 missense'))
```

Interpretation: The join condition between Variation and Gene was correctly enforced. The query traversed the `variation.isLocatedIn.gene` edge to retrieve variations linked to TP53. All returned mutations are missense substitutions, consistent with TP53's known mutation landscape in cancer genomes.

Query 3: Multi-Hop Traversal

```
1 Q: Find 5 samples that contain a mutation in the 'EGFR' gene.
2 A:
3 (rdflib.term.Literal('1310459'), rdflib.term.Literal('Lung'))
4 (rdflib.term.Literal('1098843'), rdflib.term.Literal('Lung'))
```

Interpretation: This query required a 2-hop path: Gene $\rightarrow$ Variation $\rightarrow$ Sample. Only 2 samples were returned (not 5), indicating limited EGFR coverage in the 25/75 data split. Crucially, both samples are from "Lung" tissue, which is biologically accurate—EGFR mutations are highly prevalent in lung adenocarcinomas. This validates both the technical correctness of the join and the biological fidelity of the data.

Query 4: Aggregation

```
1 Q: How many mutations are recorded for the gene 'KRAS'?
2 A:
3 (rdflib.term.Literal('5', datatype=...#integer))
```

Interpretation: The COUNT aggregation correctly computed the cardinality of mutations linked to KRAS, demonstrating that SPARQL’s aggregation operators function as expected on the materialized graph.

Overall Assessment: 14/14 queries executed successfully with semantically correct results. Zero orphan mutations were detected (Query: "Find mutations not linked to genes" returned count=0), confirming referential integrity. The few "No results found" cases (e.g., specific mutation strings) are expected given the data subset used.

5 Downstream Task 2: RAG-Based Natural Language QA

5.1 Objective and Methodology

While SPARQL provides precise querying, it requires technical expertise. To democratize KG access, we implemented a RAG system that accepts natural language questions and grounds answers in the graph’s facts.

The architecture consists of three modules:

1. **Entity Linker:** Performs keyword matching to identify entities (Genes, Primary Sites) mentioned in the user’s question
2. **Context Retriever:** Extracts a local subgraph (1-hop neighborhood) around matched entities
3. **LLM Generator:** Passes the retrieved triples to Cerebras LLM with a strict prompt: "Answer using ONLY the provided context"

5.2 Implementation

The entity linker is a simple string matching heuristic:

```
1 def find_entities(graph, question):
2     IASIS = Namespace("http://project-iasis.eu/vocab/")
3     entities = []
4
5     # Search for gene labels
6     for s, p, o in graph.triples((None, IASIS.geneLabel, None)):
7         gene_label = str(o)
8         if gene_label.lower() in question.lower():
9             entities.append((s, gene_label))
10
11     return entities
```

Listing 4: Entity Detection Logic

Context retrieval fetches all triples where the matched entity appears as subject or object:


```

1 def get_context(graph, entities):
2     context = []
3     for uri, label in entities:
4         # Outgoing edges
5         for s, p, o in graph.triples((uri, None, None)):
6             context.append((s, p, o))
7         # Incoming edges
8         for s, p, o in graph.triples((None, None, uri)):
9             context.append((s, p, o))
10    return context

```

Listing 5: Subgraph Extraction

The LLM prompt template:

```

1 system_prompt = """You are a genomics assistant. Answer the user's question
   using ONLY the provided RDF triples. If the answer is not in the context,
   state that you don't know."""
2
3 user_prompt = f"""Context:
4 {context_triples_as_string}
5
6 Question: {question}"""
7
8 response = cerebras_client.chat.completions.create(
9     model="llama3.3-70b",
10    messages=[{"role": "system", "content": system_prompt},
11              {"role": "user", "content": user_prompt}]
12 )

```

Listing 6: RAG Prompt Construction

5.3 Results and Analysis

Query 1: Single-Entity Retrieval

```

1 Q: Tell me about the gene KRAS.
2   Found Entities: ['KRAS']
3   Retrieved Context Size: 31 triples
4 A: Based on the provided RDF triples, here's what I know about the gene KRAS:
5
6 * The gene KRAS has a type of <http://project-iasis.eu/vocab/Gene>.
7 * The gene label of KRAS is "KRAS".
8 * The HGNC ID of KRAS is 6407.0.
9 * There are several variations located in the KRAS gene, including:
10 + Variation 20335507
11 + Variation 20335508
12 + Variation 20335514
13 + Variation 20335539
14 + Variation 20335548
15
16 These variations have different characteristics, such as cFormat and
   mutationDescription, but they are all located in the KRAS gene.

```

Analysis: The LLM successfully synthesized 31 raw RDF triples into a coherent summary. It correctly identified the gene's class, extracted the HGNC ID (6407.0), and enumerated linked variations by their identifiers. The phrasing "variations have different characteristics" demonstrates that the model understood the triples represent distinct mutation events, not properties of the gene itself.

Query 2: Multi-Entity Complex Query

```

1 Q: Are there any mutations in EGFR associated with Lung samples?
2 Found Entities: ['Lung', ... 'EGFR' ...]
3 Retrieved Context Size: 1534 triples
4 A: To answer this question, we need to find mutations associated with the EGFR
   gene and also find samples that are from the Lung primary site.
5
6 From the provided triples, we can see that:
7 - The EGFR gene is mentioned...
8 - However, there is no direct triple linking a variation to the EGFR gene. But,
   we do have a variation with the cFormat 'EGFR_c.2076_2090del'...
9 - This variation is not directly linked to a sample...
10
11 However, we cannot confirm if the 'EGFR_c.2076_2090del' mutation is associated
   with any of the Lung samples based on the provided triples, as there is no
   direct link between the variation and the sample.

```

Analysis: This demonstrates high fidelity to the context. The retriever found both "EGFR" (gene) and "Lung" (primary site) entities, extracting 1,534 triples (too many due to "Lung" matching many samples). Despite having evidence for both entities, the LLM correctly identified that the *specific retrieved triple set* lacked the connecting edge: (EGFR_mutation, variation_isFoundIn_sample, Lung_sample). Rather than hallucinating a connection, it explicitly stated the limitation: "we cannot confirm...based on the provided triples."

This behavior is critical for RAG system trustworthiness. The model resisted the temptation to infer or fabricate a relationship, adhering strictly to graph-grounded facts.

Overall Assessment: The RAG system successfully translated natural language queries into subgraph retrievals and synthesized responses by reasoning over structured triples. The LLM exhibited no hallucination in our test queries, validating the architecture's soundness.

6 Benefits of the Planning Approach

The RML-Planner methodology delivered measurable improvements in our implementation:

1. **Automatic Deduplication:** Despite $10\times$ replication in the input (133K records), the planner's eager DR operators reduced output to 34K unique triples, a 74% reduction. Without planning, naive execution would materialize all 133K, then deduplicate post-hoc, consuming far more memory.
2. **Source Abstraction:** The unified mapping strategy allowed seamless integration of CSV and JSON without custom ETL scripts. The planner's source-agnostic partitioning ensured both formats were processed with identical optimization logic.
3. **Execution Efficiency:** By grouping related mappings (Gene/Variation/Sample for CSV; Gene/Variation/Sample for JSON), the planner minimized file I/O. Each source was scanned exactly once per partition, rather than $3\times$ (once per entity type).
4. **Scalability:** The generated graph supports complex SPARQL queries with sub-second response times, validating that the optimization did not compromise semantic correctness for performance gains.

These benefits directly address the limitations identified in Section 1: redundant I/O is eliminated via partitioning, intermediate explosion is mitigated by eager DR, and sequential bottlenecks are resolved through bushy tree parallelization.

References

- [1] E. Iglesias, S. Jozashoori, D. Chaves-Fraga, D. Collarana, and M.-E. Vidal. *Scaling Up Knowledge Graph Creation to Large and Heterogeneous Data Sources*. arXiv preprint arXiv:2201.09694, 2022. <https://arxiv.org/pdf/2201.09694>
- [2] SDM-TIB. *RML-Planner: Optimizing the Execution Plan for Knowledge Graph Creation*. GitHub repository. <https://github.com/SDM-TIB/RML-Planner>
- [3] SDM-TIB. *SDM-Genomic Datasets*. Figshare, 2021. <https://doi.org/10.6084/m9.figshare.14838342.v1>
- [4] SDM-TIB. *IASIS-KG: Genomic Mapping Assertions*. GitHub repository. <https://github.com/SDM-TIB/IASIS-KG/tree/master/settings/mappings/genomic>
- [5] OEG-UPM. *Morph-KGC: v1.4.1*. GitHub repository. <https://github.com/oeg-upm/Morph-KGC>
- [6] SDM-TIB. *SDM-RDFizer: v4.0*. Python Package Index. <https://pypi.org/project/rdfizer/4.0/>
- [7] E. Iglesias, S. Jozashoori, and M.-E. Vidal. *Planning for KGs: Experimental Study*. GitHub repository. <https://github.com/SDM-TIB/Planning4KGC>